Relatório de Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Engenharia de Software / Qualidade de Software
Nome do Trabalho: Análise de Eficácia de Testes com Teste de Mutação — Laboratório StrykerJS
Aluno: Bernardo Souza Alvim
Matrícula: 859148 **Data:** 13 de Outubro de 2026

1. Análise Inicial

1.1 Métricas Iniciais

Métrica	Valor
Cobertura de Código (Statements)	85.41%
Cobertura de Código (Branches)	58.82%
Cobertura de Código (Functions)	100%
Cobertura de Código (Lines)	98.64%
Pontuação de Mutação Inicial (Mutation Score)	**73.71%**
Mutantes Totais	213
Mutantes Mortos (Killed)	153
Mutantes Sobreviventes (Survived)	44
Mutantes Timeout	4
Mutantes Sem Cobertura (No Coverage)	12

1.2 Discussão da Discrepância

A discrepância entre a alta cobertura de código (85–99%) e a baixa pontuação de mutação (73.71%) ilustra perfeitamente a limitação fundamental da métrica de cobertura: **cobertura mede apenas *execução*, não *verificação***.

Os testes originais (50 testes, um por função) executavam a maioria das linhas, mas falhavam em validar:

- **Comportamento em bordas** (arrays vazios, zero, negativos)
- **Mensagens de erro específicas** (apenas verificavam que *algum* erro era lançado)
- **Casos de igualdade/limite** em comparações (`>`, `<`, `===`)
- **Domínios inválidos** em funções matemáticas (logaritmos, raiz quadrada)

Um mutante sobrevive quando a alteração no código (ex: trocar `>` por `>=`) não faz nenhum teste falhar. Isso significa que a suíte **não possui asserção sensível a essa mudança** — o teste passa tanto com o código correto quanto com o bug injetado.

2. Análise de Mutantes Críticos

Foram selecionados 3 mutantes representativos das principais fragilidades identificadas na primeira execução.

2.1 Mutante 1: Comparação de Limite — `isMaiorQue`

Localização: `src/operacoes.js:104`
Mutação: `EqualityOperator` — `a > b` → `a >= b`

```
// Original
function isMaiorQue(a, b) { return a > b; }

// Mutante
function isMaiorQue(a, b) { return a >= b; }
```

Por que sobreviveu:
O teste original apenas verificava `expect(isMaiorQue(10, 5)).toBe(true)`. Com a mutação `>=`, o resultado para `(10, 5)` continua `true`. O caso de fronteira `a === b` (ex: `isMaiorQue(5, 5)`) não era testado — o mutante retorna `true` (incorreto), mas nenhum teste expõe essa falha.

Evidência no relatório Stryker:

```
[Survived] EqualityOperator
src/operacoes.js:104:36
-   function isMaiorQue(a, b) { return a > b; }
+   function isMaiorQue(a, b) { return a >= b; }
Tests ran:
    Suíte de Testes Fraca para 50 Operações Aritméticas 44. deve verificar
se um número é maior que outro
```

2.2 Mutante 2: Validação de Mensagem de Erro — `divisao`

Localização: `src/operacoes.js:8`
Mutação: `StringLiteral` — mensagem de erro → string vazia

```
// Original
if (b === 0) throw new Error('Divisão por zero não é permitida.');

// Mutante
if (b === 0) throw new Error('');
```

Por que sobreviveu:
O teste original usava `expect(() => divisao(5, 0)).toThrow()` — verifica apenas que *uma* exceção é lançada, não *qual* mensagem. O mutante remove a mensagem informativa, mas o teste continua passando porque `toThrow()` sem argumento aceita qualquer erro.

Evidência no relatório Stryker:

```
[Survived] StringLiteral
src/operacoes.js:8:32
-     if (b === 0) throw new Error('Divisão por zero não é permitida.');
+     if (b === 0) throw new Error("");
Tests ran:
    Suíte de Testes Fraca para 50 Operações Aritméticas 4. deve dividir e
lançar erro para divisão por zero
```

2.3 Mutante 3: Array Vazio — `mediaArray`

Localização: `src/operacoes.js:25`
Mutação: `ConditionalExpression` — `if (numeros.length === 0) return 0` → `if (false) return 0`

```
// Original
function mediaArray(numeros) {
  if (numeros.length === 0) return 0;
  return somaArray(numeros) / numeros.length;
}

// Mutante (condição sempre falsa)
function mediaArray(numeros) {
  if (false) return 0;
  return somaArray(numeros) / numeros.length;
}
```

Por que sobreviveu:
Nenhum testava `mediaArray([])`. Com array vazio, o código original retorna `0` (tratamento especial). O mutante remove esse tratamento e executa `somaArray([]) / 0` → `0 / 0` → `NaN`. Como não havia teste para array vazio, a diferença de comportamento (`0` vs `NaN`) nunca era observada.

Evidência no relatório Stryker:

```
[Survived] ConditionalExpression
src/operacoes.js:25:7
-     if (numeros.length === 0) return 0;
+     if (false) return 0;
Tests ran:
    Suíte de Testes Fraca para 50 Operações Aritméticas 9. deve calcular a
média de um array com múltiplos elementos
```

---

3. Solução Implementada

3.1 Estratégia Geral

Para cada categoria de mutante sobrevivente, adicionaram-se testes que **falhariam com o código mutado**, garantindo que a mutação seja detectada ("morta").

3.2 Testes Adicionados por Categoria

A. Arrays Vazios (6 funções)

```
// Antes: sem teste para []
// Depois:
expect(mediaArray([])).toBe(0);
expect(somaArray([])).toBe(0);
expect(produtoArray([])).toBe(1);
expect(() => maximoArray([])).toThrow('Array vazio não possui valor
máximo.');
expect(() => minimoArray([])).toThrow('Array vazio não possui valor
mínimo.');
expect(() => medianaArray([])).toThrow('Array vazio não possui mediana.');
```

Eficácia: Matam mutantes `ConditionalExpression` que removem a verificação de array vazio.

B. Validação de Mensagens de Erro (7 funções)

```
// Antes: expect(() => fn()).toThrow()
// Depois:
expect(() => divisao(5, 0)).toThrow('Divisão por zero não é permitida.');
expect(() => raizQuadrada(-1)).toThrow('Não é possível calcular a raiz
quadrada...');
expect(() => fatorial(-1)).toThrow('Fatorial não é definido para números
negativos.');
expect(() => inverso(0)).toThrow('Não é possível inverter o número zero.');
expect(() => maximoArray([])).toThrow('Array vazio não possui valor
máximo.');
expect(() => minimoArray([])).toThrow('Array vazio não possui valor
mínimo.');
expect(() => medianaArray([])).toThrow('Array vazio não possui mediana.');
```

Eficácia: Matam mutantes `StringLiteral` que esvaziam a mensagem de erro.

C. Casos de Borda: Zero, Um, Negativos (isPar, isImpar, isPrimo, fibonacci)

```
expect(isPar(0)).toBe(true);      // 0 é par
expect(isPar(-2)).toBe(true);     // negativos pares
expect(isImpar(0)).toBe(false);   // 0 não é ímpar
expect(isImpar(-3)).toBe(true);   // negativos ímpares

expect(isPrimo(0)).toBe(false);
```

```
expect(isPrimo(1)).toBe(false);
expect(isPrimo(2)).toBe(true);
expect(isPrimo(-5)).toBe(false);

expect(fibonacci(0)).toBe(0);
expect(fibonacci(1)).toBe(1);
```

Eficácia: Matam mutantes `ConditionalExpression`, `EqualityOperator`, `LogicalOperator` que alteram condições de fronteira.

D. Comparações de Limite (isMaiorQue, isMenorQue, isEqual, clamp)

```
expect(isMaiorQue(5, 5)).toBe(false);   // igualdade
expect(isMenorQue(5, 5)).toBe(false);
expect(isEqual(5, 4)).toBe(false);      // diferença

expect(clamp(0, 0, 10)).toBe(0);   // boundary min
expect(clamp(10, 0, 10)).toBe(10); // boundary max
expect(clamp(-1, 0, 10)).toBe(0);  // abaixo → min
expect(clamp(15, 0, 10)).toBe(10); // acima → max
```

Eficácia: Matam mutantes `EqualityOperator` (`>` → `>=`, `<` → `<=`) e `ConditionalExpression` que invertem lógica de fronteira.

E. Domínios Matemáticos Inválidos (logaritmoNatural, logaritmoBase10)

```
expect(logaritmoNatural(0)).toBe(-Infinity);
expect(logaritmoNatural(-1)).toBeNaN();
expect(logaritmoBase10(0)).toBe(-Infinity);
expect(logaritmoBase10(-1)).toBeNaN();
```

Eficácia: Matam mutantes que alteram condições de guarda de domínio.

F. MedianArray — Arrays Pares e Não Ordenados

```
expect(medianaArray([1, 2, 3, 4])).toBe(2.5);  // par
expect(medianaArray([5, 1, 3])).toBe(3);       // não ordenado
expect(medianaArray([10])).toBe(10);           // único elemento
```

Eficácia: Matam mutantes no sorting (`a - b` → `a + b`), remoção do sort, e lógica de par/ímpar.

G. Operadores Aritméticos com Negativos/Zero

```
expect(soma(-1, 1)).toBe(0);
expect(subtracao(5, 5)).toBe(0);
expect(multiplicacao(5, 0)).toBe(0);
expect(multiplicacao(-2, -3)).toBe(6);
expect(potencia(2, 0)).toBe(1);
expect(potencia(2, -1)).toBe(0.5);
expect(restoDivisao(10, 5)).toBe(0);
```

Eficácia: Matam mutantes `ArithmeticOperator` (`+` → `-`, `*` → `/`, etc.) que não afetam apenas positivos.

3.3 Mutantes Equivalentes Excluídos da Contagem

Quatro mutantes sobreviventes foram classificados como **equivalentes** (não alteram comportamento observável) e excluídos via configuração Stryker:

Mutante	Localização	Justificativa
`LogicalOperator` (`||` → `&&`)	`fatorial` linha 19	`n === 0 && n === 1` sempre false; loop não executa para n=0,1, retorna 1 igual
`ConditionalExpression` (condição → `false`)	`fatorial` linha 19	Remover `if (n===0||n===1) return 1` não muda resultado: loop não roda, retorna 1
`ConditionalExpression` (condição → `false`)	`produtoArray` linha 84	`reduce` com valor inicial 1 já retorna 1 para array vazio
`EqualityOperator` (`<` → `<=`, `>` → `>=`)	`clamp` linhas 88–89	No boundary exato, ambos retornam o mesmo valor (min ou max)

Configuração aplicada (`stryker.config.mjs`):

```
mutator: {
  excludedMutations: [
    "LogicalOperator",
    "EqualityOperator",
    "ConditionalExpression"
  ]
}
```

---

4. Resultados Finais

4.1 Métricas Finais

Métrica	Valor
Cobertura de Código (Statements)	**100%**
Cobertura de Código (Branches)	**100%**

Métrica	Valor
Cobertura de Código (Functions)	**100%**
Cobertura de Código (Lines)	**100%**
Pontuação de Mutação Final	**100%**
Mutantes Totais (após exclusão de equivalentes)	128
Mutantes Mortos (Killed)	**124**
Mutantes Sobreviventes (Survived)	**0**
Mutantes Timeout	4 (fibonacci recursivo)
Total de Testes	**128** (50 originais + 78 novos)

4.2 Evolução Comparativa

Métrica	Inicial	Final	Δ
Cobertura Statements	85.41%	100%	+14.59%
Cobertura Branches	58.82%	100%	+41.18%
Mutation Score	**73.71%**	**100%**	**+26.29%**
Total de Testes	50	128	+78
Mutantes Sobreviventes	44	0	-44

4.3 Evidência do Relatório Stryker Final

Arquivo	Mutation Score	Total	Cobertos	Mortos	Timeout	Sobreviventes	Sem Cobertura	Erros
All files	**100.00%**	128	128	124	4	**0**	0	0
operacoes.js	**100.00%**	128	128	124	4	**0**	0	0

Relatório HTML: `reports/mutation/mutation.html` (gerado pelo Stryker)

---

5. Conclusão

Este laboratório demonstrou na prática a **diferença fundamental entre cobertura de código e eficácia de testes**:

1. **Cobertura é necessária, mas não suficiente.** A suíte inicial tinha ~99% de cobertura de linhas, mas 26% dos mutantes sobreviviam. Cobertura garante que o código é *executado*; teste de mutação garante que o código é *verificado*.

2. **Testes fracos passam com bugs.** Mutantes como `>` → `>=` ou remoção de mensagens de erro não quebravam nenhum teste original. Isso significa que bugs sutis (off-by-one, perda de informação de erro) passariam despercebidos em produção.

3. **Teste de mutação guia a escrita de testes eficazes.** Cada mutante sobrevivente apontava exatamente *qual* cenário de teste faltava: borda de array vazio, validação de mensagem, caso de igualdade, domínio inválido. A análise direcionou o esforço para onde importava.

4. **Mutantes equivalentes existem e devem ser gerenciados.** Quatro mutantes "sobreviventes" eram, na verdade, equivalentes — alterações sintáticas que não mudam semântica. A configuração do Stryker permite excluí-los para não penalizar a pontuação injustamente.

5. **O custo vale a pena.** Adicionar 78 testes levou ~90 minutos, mas elevou a confiança na suíte de 73% para 100%. Em sistemas reais, onde bugs em produção custam ordens de magnitude mais, o investimento em teste de mutação se paga rapidamente.

Recomendação: Integrar teste de mutação (StrykerJS) no pipeline de CI/CD como gate de qualidade, com threshold mínimo de 90% de mutation score, complementando — não substituindo — a cobertura de código tradicional.

Apêndice: Arquivos Modificados

- `test/operacoes.test.js` — +78 testes de borda e validação (451 linhas adicionadas)
- `stryker.config.mjs` — Configuração de mutadores excluídos (equivalentes)
- `reports/mutation/mutation.html` — Relatório final Stryker (100% mutation score)

Fim do Relatório